

Private Service Connect 介面代管服務

來源：<https://codelabs.developers.google.com/codelabs/psc-interface-mgd-service?hl=zh-tw>

步驟數：18

在這個教學課程中，您將瞭解如何設定及驗證 Private Service Connect 介面，以便透過虛擬私有雲對等互連存取服務。

目錄

1. 簡介
2. 事前準備
3. 消費者設定
4. 啟用 IAP
5. 建立消費者 VM 執行個體
6. Private Service Connect 網路連結
7. 在消費者和後端虛擬私有雲之間建立虛擬私有雲對等互連
8. 設定 Producer
9. 建立生產者防火牆規則
10. 建立 Private Service Connect 介面
11. 11. 建立非代管執行個體群組，並將 rabbit 和 fox 新增至該群組
12. 12. 建立 TCP 健康狀態檢查、後端服務、轉送規則和防火牆
13. 13. 為 PSC 介面建立 Linux IP 資料表 - rabbit
14. 14. 為 PSC 介面建立 Linux IP 資料表 - fox
15. 15. 更新路由表
16. 16. 確認從 tiger 到 cosmo 的連線是否成功
17. 17. 清理
18. 18. 恭喜

步驟 1 · 約 0 分鐘

Private Service Connect 介面代管服務

程式碼研究室簡介

subject上次更新時間：3月 27, 2026

account_circle作者：Deepak Michael, Nick De Cristofaro

1. 簡介

Private Service Connect 介面是一種資源，可讓供應商虛擬私有雲 (VPC) 網路啟動與用戶虛擬私有雲網路中各種目的地的連線。供應商和用戶網路可以位於不同專案和機構中。

如果網路連結接受來自 Private Service Connect 介面的連線，Google Cloud 會從網路連結指定的消費者子網路，為介面分配 IP 位址。消費者和生產者網路已連線，可使用內部 IP 位址通訊。

網路連結與 Private Service Connect 介面之間的連線，類似於 Private Service Connect 端點與服務連結之間的連線，但有兩項主要差異：

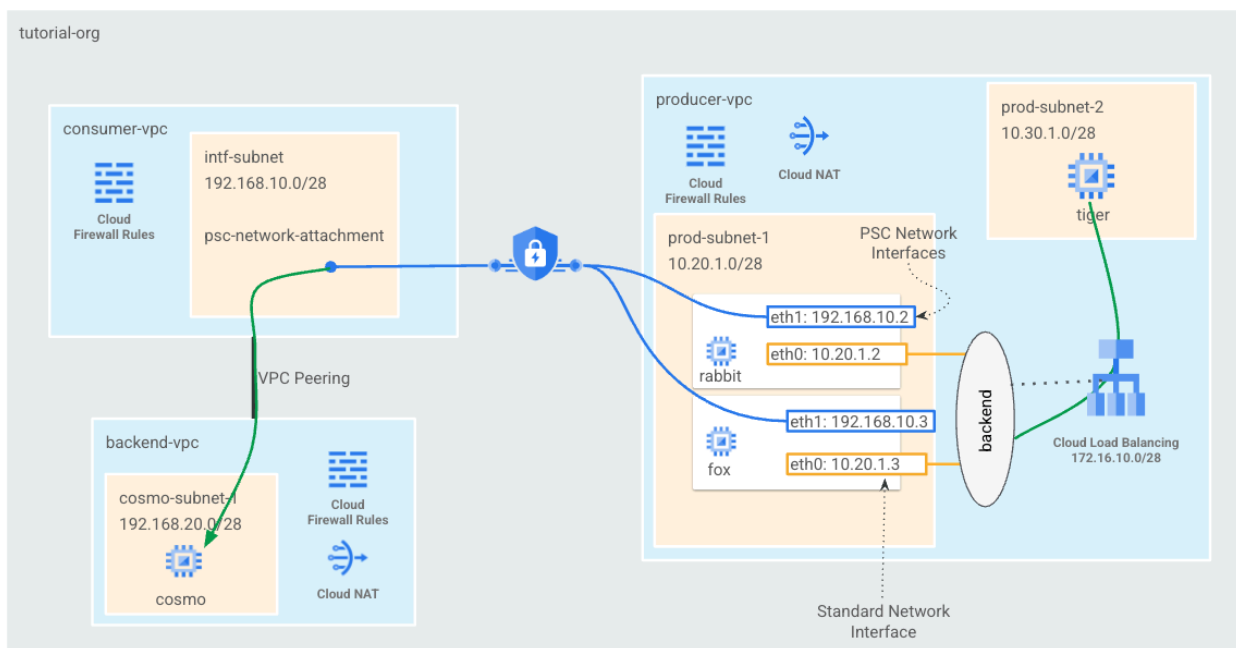
- 網路連結可讓供應商網路啟動與消費者網路的連線 (代管服務輸出)，而端點則可讓消費者網路啟動與供應商網路的連線 (代管服務輸入)。
- Private Service Connect 介面連線是可遞移的。也就是說，生產端網路可以與連線至消費端網路的其他網路通訊。

建構項目

您會在消費者 VPC 中建立單一 psc-network-attachment，產生兩個 PSC 介面，做為 L4 內部負載平衡器的後端。從供應商虛擬私有雲，tiger 會將 curl 傳送至 backend-vpc 中的 cosmo。在生產者虛擬私有雲中，您將建立靜態路徑，將目的地流量 192.168.20.0/28 的下一個躍點設為內部負載平衡器，該負載平衡器會利用後端和後續的 PSC 介面，將流量路由至 cosmo。請參閱圖 1 瞭解概況。

使用[私人服務存取權](#)時，如果 Google 代管服務與客戶虛擬私有雲進行虛擬私有雲對等互連，也可以採用相同方法。

圖 1



注意：本教學課程會根據圖示拓撲提供設定和驗證步驟，請視貴機構的需求修改程序。

課程內容

- 如何建立網路連結
- 生產者如何使用網路附件建立 PSC 介面做為後端
- 如何使用內部負載平衡器做為下一個躍點，建立從製作人到消費者的通訊
- 如何透過虛擬私有雲對等互連，允許從供應商 VM (tiger) 存取用戶端 VM (cosmo)

軟硬體需求

- Google Cloud 專案
 - IAM 權限
 - [Compute 網路管理員](#) (roles/compute.networkAdmin)
 - [Compute 執行個體管理員](#) (roles/compute.instanceAdmin)
 - [Compute 安全管理員](#) (roles/compute.securityAdmin)
-

步驟 2 · 約 2 分鐘

2. 事前準備

更新專案以支援教學課程

本教學課程會使用 `$variables`，協助您在 Cloud Shell 中實作 gcloud 設定。

在 Cloud Shell 中執行下列操作：



```
gcloud config list project
gcloud config set project [YOUR-PROJECT-NAME]
projectid=YOUR-PROJECT-NAME
echo $projectid
```

步驟 3 · 約 10 分鐘

3. 消費者設定

建立消費者虛擬私有雲

在 Cloud Shell 中執行下列操作：

〇〇〇

```
gcloud compute networks create consumer-vpc --project=$projectid --subnet-mode=custom
```

建立 Private Service Connect 網路連結子網路

在 Cloud Shell 中執行下列操作：

〇〇〇

```
gcloud compute networks subnets create intf-subnet --project=$projectid --range=192.168.10.0/28 --network=consumer-vpc --region=us-central1
```

建立後端虛擬私有雲

在 Cloud Shell 中執行下列操作：

〇〇〇

```
gcloud compute networks create backend-vpc --project=$projectid --subnet-mode=custom
```

建立後端虛擬私有雲子網路

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks subnets create cosmo-subnet-1 --project=$projectid --range=192.168.20.0/28 --network=backend-vpc --region=us-central1
```

建立後端虛擬私有雲防火牆規則

在 Cloud Shell 中，為從 psc-network-attachment 子網路到 cosmo 的流量建立輸入規則

```
gcloud compute firewall-rules create allow-ingress-to-cosmo \
  --network=backend-vpc \
  --action=ALLOW \
  --rules=ALL \
  --direction=INGRESS \
  --priority=1000 \
  --source-ranges="192.168.10.0/28" \
  --destination-ranges="192.168.20.0/28" \
  --enable-logging
```

Cloud Router 和 NAT 設定

本教學課程使用 Cloud NAT 安裝軟體套件，因為 VM 執行個體沒有公開 IP 位址。Cloud NAT 可讓具有私人 IP 位址的 VM 存取網際網路。

在 Cloud Shell 中建立 Cloud Router。

```
gcloud compute routers create cloud-router-for-nat --network backend-vpc --region us-central1
```

在 Cloud Shell 中建立 NAT 閘道。

```
gcloud compute routers nats create cloud-nat-us-central1 --router=cloud-router-for-nat --  
auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges --region us-central1
```

步驟 4 · 約 0 分鐘

4. 啟用 IAP

如要允許 IAP 連線至您的 VM 執行個體，請根據以下條件建立防火牆規則：

- 套用至所有您希望能透過 IAP 存取的 VM 執行個體。
- 允許來自 IP 範圍 35.235.240.0/20 的輸入流量。這個範圍包含 IAP 用於 TCP 轉送的所有 IP 位址。

在 Cloud Shell 中，建立 IAP 防火牆規則。

```
gcloud compute firewall-rules create ssh-iap-consumer \  
  --network backend-vpc \  
  --allow tcp:22 \  
  --source-ranges=35.235.240.0/20
```

步驟 5 · 約 0 分鐘

5. 建立消費者 VM 執行個體

在 Cloud Shell 中建立消費者 VM 執行個體 cosmo

```
gcloud compute instances create cosmo \  
  --project=$projectid \  
  --machine-type=e2-micro \  
  --image-family debian-11 \  
  --no-address \  
  --image-project debian-cloud \  
  --zone us-central1-a \  
  --subnet=cosmo-subnet-1 \  
  --metadata startup-script="#! /bin/bash  
  sudo apt-get update  
  sudo apt-get install tcpdump  
  sudo apt-get install apache2 -y  
  sudo service apache2 restart  
  echo 'Welcome to cosmo's backend server !!' | tee /var/www/html/index.html  
  EOF"
```

取得並儲存執行個體的 IP 位址：

在 Cloud Shell 中，對 cosmo VM 執行個體執行說明。

```
gcloud compute instances describe cosmo --zone=us-central1-a | grep networkIP:
```

步驟 6 · 約 5 分鐘

6. Private Service Connect 網路連結

網路連結是區域資源，代表 Private Service Connect 介面的用戶端。您會將單一子網路與網路連結建立關聯，而生產端會從該子網路將 IP 指派給 Private Service Connect 介面。子網路必須與網路連結位於同一地區。網路連結必須與生產者服務位於相同區域。

建立網路連結

在 Cloud Shell 中建立網路連結。

```
gcloud compute network-attachments create psc-network-attachment \
  --region=us-central1 \
  --connection-preference=ACCEPT_MANUAL \
  --producer-accept-list=$projectid \
  --subnets=intf-subnet
```

列出網路連結

在 Cloud Shell 中列出網路連結。

```
gcloud compute network-attachments list
```

說明網路連結

在 Cloud Shell 中，說明網路附件。

```
gcloud compute network-attachments describe psc-network-attachment --region=us-central1
```

請記下 psc-network-attachment URI，供應商建立 Private Service Connect 介面時會用到。範例如下：

```
user$ gcloud compute network-attachments describe psc-network-attachment --region=us-central1
connectionPreference: ACCEPT_MANUAL
creationTimestamp: '2023-06-07T11:27:33.116-07:00'
fingerprint: 8SDsvG6TfYQ=
id: '5014253525248340730'
kind: compute#networkAttachment
name: psc-network-attachment
network: https://www.googleapis.com/compute/v1/projects/$projectid/global/networks/consumer-
vpc
producerAcceptLists:
- $projectid
region: https://www.googleapis.com/compute/v1/projects/$projectid/regions/us-central1
selfLink: https://www.googleapis.com/compute/v1/projects/$projectid/regions/us-
central1/networkAttachments/psc-network-attachment
subnetworks:
- https://www.googleapis.com/compute/v1/projects/$projectid/regions/us-
central1/subnetworks/intf-subnet
```

步驟 7 · 約 10 分鐘

7. 在消費者和後端虛擬私有雲之間建立虛擬私有雲對等互連

您將在消費者和後端 VPC 之間建立 VPC 對等互連連線。這項功能會複製 Google 為代管服務建立與客戶虛擬私有雲的連線能力方式，以及網路連線的跨機構對等互連。您必須從每個 VPC 設定 VPC 對等互連。

從消費者虛擬私有雲到後端虛擬私有雲的對等互連

從消費者端建立虛擬私有雲對等互連連線至後端虛擬私有雲

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks peerings create consumer-to-backend-vpc \  
  --network=consumer-vpc \  
  --peer-project=$projectid \  
  --peer-network=backend-vpc \  
  --stack-type=IPV4_ONLY
```

從後端建立虛擬私有雲對等互連連線至消費者虛擬私有雲

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks peerings create backend-to-consumer-vpc \  
  --network=backend-vpc \  
  --peer-project=$projectid \  
  --peer-network=consumer-vpc \  
  --stack-type=IPV4_ONLY
```

驗證虛擬私有雲對等互連狀態詳細資料

在 Cloud Shell 中，確認 VPC 對等互連處於「Active」和「Connected」狀態。

```
gcloud compute networks peerings list
```

範例：

```
user@cloudshell$ gcloud compute networks peerings list
NAME: backend-to-consumer-vpc
NETWORK: backend-vpc
PEER_PROJECT: $projectid
PEER_NETWORK: consumer-vpc
STACK_TYPE: IPV4_ONLY
PEER_MTU:
IMPORT_CUSTOM_ROUTES: False
EXPORT_CUSTOM_ROUTES: False
STATE: ACTIVE
STATE_DETAILS: [2023-06-07T11:42:27.634-07:00]: Connected.

NAME: consumer-to-backend-vpc
NETWORK: consumer-vpc
PEER_PROJECT: $projectid
PEER_NETWORK: backend-vpc
STACK_TYPE: IPV4_ONLY
PEER_MTU:
IMPORT_CUSTOM_ROUTES: False
EXPORT_CUSTOM_ROUTES: False
STATE: ACTIVE
STATE_DETAILS: [2023-06-07T11:42:27.634-07:00]: Connected.
```

步驟 8 · 約 10 分鐘

8. 設定 Producer

建立生產者虛擬私有雲

在 Cloud Shell 中執行下列操作：

```
gcloud compute networks create producer-vpc --project=$projectid --subnet-mode=custom
```

建立生產端子網路

在 Cloud Shell 中，建立用於 psc 介面 vNIC0 的子網路

```
gcloud compute networks subnets create prod-subnet --project=$projectid --range=10.20.1.0/28  
--network=producer-vpc --region=us-central1
```

在 Cloud Shell 中，建立執行個體 tiger 用的子網路。

```
gcloud compute networks subnets create prod-subnet-2 --project=$projectid --  
range=10.30.1.0/28 --network=producer-vpc --region=us-central1
```

在 Cloud Shell 中，建立內部負載平衡器使用的子網路。

```
gcloud compute networks subnets create prod-subnet-3 --project=$projectid --  
range=172.16.10.0/28 --network=producer-vpc --region=us-central1
```

Cloud Router 和 NAT 設定

本教學課程使用 Cloud NAT 安裝軟體套件，因為 VM 執行個體沒有公開 IP 位址。Cloud NAT 可讓具有私人 IP 位址的 VM 存取網際網路。

在 Cloud Shell 中建立 Cloud Router。

```
gcloud compute routers create cloud-router-for-nat-producer --network producer-vpc --region  
us-central1
```

在 Cloud Shell 中建立 NAT 閘道。

```
gcloud compute routers nats create cloud-nat-us-central1-producer --router=cloud-router-for-  
nat-producer --auto-allocate-nat-external-ips --nat-all-subnet-ip-ranges --region us-central1
```

啟用 IAP

如要允許 IAP 連線至您的 VM 執行個體，請根據以下條件建立防火牆規則：

- 套用至所有您希望能透過 IAP 存取的 VM 執行個體。
- 允許來自 IP 範圍 35.235.240.0/20 的輸入流量。這個範圍包含 IAP 用於 TCP 轉送的所有 IP 位址。

在 Cloud Shell 中，建立 IAP 防火牆規則。

```
gcloud compute firewall-rules create ssh-iap-producer \  
  --network producer-vpc \  
  --allow tcp:22 \  
  --source-ranges=35.235.240.0/20
```

建立 Producer VM 執行個體

在 Cloud Shell 中，建立取用者 VM 執行個體 tiger

```
gcloud compute instances create tiger \  
  --project=$projectid \  
  --machine-type=e2-micro \  
  --image-family debian-11 \  
  --no-address \  
  --image-project debian-cloud \  
  --zone us-central1-a \  
  --subnet=prod-subnet-2 \  
  --metadata startup-script="#!/bin/bash  
  sudo apt-get update  
  sudo apt-get install tcpdump"
```

步驟 9 · 約 0 分鐘

9. 建立生產者防火牆規則

在生產者虛擬私有雲中，建立允許從 prod-subnet-2 與生產者虛擬私有雲中所有執行個體通訊的輸入防火牆規則。

在 Cloud Shell 中，建立生產者防火牆規則。

```
gcloud compute --project=$projectid firewall-rules create allow-tiger-ingress --  
direction=INGRESS --priority=1000 --network=producer-vpc --action=ALLOW --rules=all --source-  
ranges=10.30.1.0/28 --enable-logging
```

10. 建立 Private Service Connect 介面

Private Service Connect 介面是一種資源，可讓供應商虛擬私有雲 (VPC) 網路啟動與用戶虛擬私有雲網路中各種目的地的連線。供應商和用戶網路可以位於不同專案和機構中。

如果網路連結接受來自 Private Service Connect 介面的連線，Google Cloud 會從網路連結指定的消費者子網路，為介面分配 IP 位址。消費者和生產者網路已連線，可使用內部 IP 位址通訊。

在本教學課程中，您將建立兩個執行個體，並使用 Private Service Connect 網路附件做為內部負載平衡器的後端。

在 Cloud Shell 中，建立 Private Service Connect 介面 (rabbit)，並插入先前從網路連結說明輸出內容中識別的 **psc-network-attachment URI**。

```
gcloud compute instances create rabbit --zone us-central1-a --machine-type=f1-micro --can-ip-forward --network-interface subnet=prod-subnet,network=producer-vpc,no-address --network-interface network-attachment=https://www.googleapis.com/compute/v1/projects/$projectid/regions/us-central1/networkAttachments/psc-network-attachment --metadata startup-script="#! /bin/bash\n  sudo apt-get update\n  sudo apt-get install tcpdump\n  sudo apt-get install apache2 -y\n  sudo service apache2 restart"
```

在 Cloud Shell 中，建立 Private Service Connect 介面 (fox)，並插入先前從網路連結說明輸出內容中識別的 **psc-network-attachment URI**。

```
gcloud compute instances create fox --zone us-central1-a --machine-type=f1-micro --can-ip-forward --network-interface subnet=prod-subnet,network=producer-vpc,no-address --network-interface network-attachment=https://www.googleapis.com/compute/v1/projects/$projectid/regions/us-central1/networkAttachments/psc-network-attachment --metadata startup-script="#! /bin/bash\n  sudo apt-get update\n  sudo apt-get install tcpdump\n  sudo apt-get install apache2 -y\n  sudo service apache2 restart"
```

多重 NIC 驗證

確認 PSC 介面已設定適當的 IP 位址。vNIC0 會使用生產者 prod-subnet (10.20.1.0/28)，vNIC1 則會使用消費者 intf-subnet (192.168.10.0/28)。

```
gcloud compute instances describe rabbit --zone=us-central1-a | grep networkIP:\n\ngcloud compute instances describe fox --zone=us-central1-a | grep networkIP:
```

範例：

```
user$ gcloud compute instances describe rabbit --zone=us-central1-a | grep networkIP:  
networkIP: 10.20.1.2  
networkIP: 192.168.10.2
```

```
user$ gcloud compute instances describe fox --zone=us-central1-a | grep networkIP:  
networkIP: 10.20.1.3  
networkIP: 192.168.10.3
```

11. 建立非代管執行個體群組，並將 rabbit 和 fox 新增至該群組

在接下來的章節中，您將建立非代管執行個體群組，其中包含 PSC 介面執行個體 rabbit 和 fox。

在 Cloud Shell 中，建立非代管執行個體群組。

```
gcloud compute instance-groups unmanaged create psc-interface-instances-ig --  
project=$projectid --zone=us-central1-a
```

在 Cloud Shell 中，將 fox 和 rabbit 執行個體新增至執行個體群組。

```
gcloud compute instance-groups unmanaged add-instances psc-interface-instances-ig --  
project=$projectid --zone=us-central1-a --instances=fox,rabbit
```

12. 建立 TCP 健康狀態檢查、後端服務、轉送規則和防火牆

在 Cloud Shell 中，建立後端健康狀態檢查。

```
gcloud compute health-checks create http hc-http-80 --port=80
```

在 Cloud Shell 中建立後端服務

```
gcloud compute backend-services create psc-interface-backend --load-balancing-scheme=internal  
--protocol=tcp --region=us-central1 --health-checks=hc-http-80
```

```
gcloud compute backend-services add-backend psc-interface-backend --region=us-central1 --  
instance-group=psc-interface-instances-ig --instance-group-zone=us-central1-a
```

在 Cloud Shell 中建立轉送規則

```
gcloud compute forwarding-rules create psc-ilb --region=us-central1 --load-balancing-  
scheme=internal --network=producer-vpc --subnet=prod-subnet-3 --address=172.16.10.10 --ip-  
protocol=TCP --ports=all --backend-service=psc-interface-backend --backend-service-region=us-  
central1
```

在 Cloud Shell 中建立防火牆規則，啟用後端健康狀態檢查

```
gcloud compute firewall-rules create ilb-health-checks --allow tcp:80,tcp:443 --network  
producer-vpc --source-ranges 130.211.0.0/22,35.191.0.0/16
```

13. 為 PSC 介面建立 Linux IP 資料表 - rabbit

在 PSC 介面執行個體中，設定 Linux IP 表格，允許生產者與消費者子網路通訊。

找出 Private Service Connect 介面的客體 OS 名稱

如要設定路由，您必須知道 Private Service Connect 介面的客體 OS 名稱，這與 Google Cloud 中的介面名稱不同。

使用 Cloud Shell 中的 IAP 登入 psc-interface VM (rabbit)。

```
gcloud compute ssh rabbit --project=$projectid --zone=us-central1-a --tunnel-through-iap
```

在 Cloud Shell 中取得 psc-interface 執行個體的 IP 位址

```
ip a
```

範例：

```
user@rabbit:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1460 qdisc pfifo_fast state UP group default
    qlen 1000
    link/ether 42:01:0a:14:01:02 brd ff:ff:ff:ff:ff:ff
    altname enp0s4
    inet 10.20.1.2/32 brd 10.20.1.2 scope global dynamic ens4
        valid_lft 59396sec preferred_lft 59396sec
    inet6 fe80::4001:aff:fe14:102/64 scope link
        valid_lft forever preferred_lft forever
3: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1460 qdisc pfifo_fast state UP group default
    qlen 1000
    link/ether 42:01:c0:a8:0a:02 brd ff:ff:ff:ff:ff:ff
    altname enp0s5
    inet 192.168.10.2/32 brd 192.168.10.2 scope global dynamic ens5
        valid_lft 66782sec preferred_lft 66782sec
    inet6 fe80::4001:c0ff:fea8:a02/64 scope link
        valid_lft forever preferred_lft forever
```

找出 PSC 介面的閘道 IP

在網路介面清單中，找出並儲存與 Private Service Connect 介面 IP 位址相關聯的介面名稱，例如 **ens5 (vNIC1)**

如要設定路由，您必須知道 Private Service Connect 介面預設閘道的 IP 位址

在 Cloud Shell 中，由於 PSC 介面與 vNIC1 相關聯，因此我們將使用 **1**。

```
curl http://metadata.google.internal/computeMetadata/v1/instance/network-interfaces/1/gateway
-H "Metadata-Flavor: Google" && echo
```

範例會產生預設閘道 192.168.10.1

```
user@rabbit:~$ curl http://metadata.google.internal/computeMetadata/v1/instance/network-
interfaces/1/gateway -H "Metadata-Flavor: Google" && echo
192.168.10.1
```

新增消費者子網路的路徑

您必須為連線至 Private Service Connect 介面的每個消費者子網路，將路徑新增至 Private Service Connect 介面的預設閘道。這可確保傳往消費者網路的流量會從 Private Service Connect 介面輸出。

驗證路由表

在 Cloud Shell 中驗證目前的路由。

```
ip route show
```

Example.

```
user@rabbit:~$ ip route show
default via 10.20.1.1 dev ens4
10.20.1.0/28 via 10.20.1.1 dev ens4
10.20.1.1 dev ens4 scope link
192.168.10.0/28 via 192.168.10.1 dev ens5
192.168.10.1 dev ens5 scope link
```

在 Cloud Shell 中，將路徑新增至 cosmo-subnet-1

```
sudo ip route add 192.168.20.0/28 via 192.168.10.1 dev ens5
```

驗證路由表

在 Cloud Shell 中驗證更新後新增的路徑。

```
ip route show
```

Example.

```
user@rabbit:~$ ip route show
default via 10.20.1.1 dev ens4
10.20.1.0/28 via 10.20.1.1 dev ens4
10.20.1.1 dev ens4 scope link
192.168.10.0/28 via 192.168.10.1 dev ens5
192.168.10.1 dev ens5 scope link
192.168.20.0/28 via 192.168.10.1 dev ens5
```

建立 IP 資料表規則

在 Cloud Shell 中驗證目前的 IP 表格。

```
sudo iptables -t nat -L -n -v
```

範例：

```
user@rabbit:~$ sudo iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source      destination

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source      destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source      destination

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source      destination
```

在 Cloud Shell 中更新 IP 資料表

```
sudo iptables -t nat -A POSTROUTING -o ens5 -j MASQUERADE
sudo sysctl net.ipv4.ip_forward=1
```

在 Cloud Shell 中驗證更新後的 IP 資料表。

```
sudo iptables -t nat -L -n -v
```

範例：

```
user@rabbit:~$ sudo iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source        destination

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source        destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source        destination

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source        destination
    0      0 MASQUERADE  all  --  *       ens5     0.0.0.0/0     0.0.0.0/0
```

步驟 14 · 約 2 分鐘

14. 為 PSC 介面建立 Linux IP 資料表 - fox

在 PSC 介面執行個體中，設定 Linux IP 表格，允許生產者與消費者子網路通訊。

找出 Private Service Connect 介面的客體 OS 名稱

如要設定路由，您必須知道 Private Service Connect 介面的客體 OS 名稱，這與 Google Cloud 中的介面名稱不同。

開啟新的 Cloud Shell 分頁，然後更新專案設定。

在 Cloud Shell 中執行下列操作：

```
gcloud config list project
gcloud config set project [YOUR-PROJECT-NAME]
projectid=YOUR-PROJECT-NAME
echo $projectid
```

使用 Cloud Shell 中的 IAP 登入 psc-interface VM (fox)。

```
gcloud compute ssh fox --project=$projectid --zone=us-central1-a --tunnel-through-iap
```

在 Cloud Shell 中取得 psc-interface 執行個體的 IP 位址

```
ip a
```

範例：

```

user@fox:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: ens4: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1460 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 42:01:0a:14:01:03 brd ff:ff:ff:ff:ff:ff
    altname enp0s4
    inet 10.20.1.3/32 brd 10.20.1.3 scope global dynamic ens4
        valid_lft 65601sec preferred_lft 65601sec
    inet6 fe80::4001:aff:fe14:103/64 scope link
        valid_lft forever preferred_lft forever
3: ens5: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1460 qdisc pfifo_fast state UP group default qlen 1000
    link/ether 42:01:c0:a8:0a:03 brd ff:ff:ff:ff:ff:ff
    altname enp0s5
    inet 192.168.10.3/32 brd 192.168.10.3 scope global dynamic ens5
        valid_lft 63910sec preferred_lft 63910sec
    inet6 fe80::4001:c0ff:fea8:a03/64 scope link
        valid_lft forever preferred_lft forever

```

找出 PSC 介面的閘道 IP

在網路介面清單中，找出並儲存與 **Private Service Connect** 介面 IP 位址相關聯的介面名稱，例如 **ens5 (vNIC1)**

如要設定路由，您必須知道 Private Service Connect 介面預設閘道的 IP 位址

在 Cloud Shell 中，由於 PSC 介面與 vNIC1 相關聯，因此我們將使用 **1**。

```

curl http://metadata.google.internal/computeMetadata/v1/instance/network-interfaces/1/gateway
-H "Metadata-Flavor: Google" && echo

```

範例會產生預設閘道 192.168.10.1

```

user@fox:~$ curl http://metadata.google.internal/computeMetadata/v1/instance/network-
interfaces/1/gateway -H "Metadata-Flavor: Google" && echo
192.168.10.1

```

新增消費者子網路的路徑

您必須為連線至 Private Service Connect 介面的每個消費者子網路，將路徑新增至 Private Service Connect 介面的預設閘道。這可確保傳往消費者網路的流量會從 Private Service Connect 介面輸出。

驗證路由表

在 Cloud Shell 中驗證目前的路由。

```
ip route show
```

Example.

```
user@fox:~$ ip route show
default via 10.20.1.1 dev ens4
10.20.1.0/28 via 10.20.1.1 dev ens4
10.20.1.1 dev ens4 scope link
192.168.10.0/28 via 192.168.10.1 dev ens5
192.168.10.1 dev ens5 scope link
```

在 Cloud Shell 中，將路徑新增至 cosmo-subnet-1

```
sudo ip route add 192.168.20.0/28 via 192.168.10.1 dev ens5
```

驗證路由表

在 Cloud Shell 中驗證更新後新增的路徑。

```
ip route show
```

Example.

```
user@fox:~$ ip route show
default via 10.20.1.1 dev ens4
10.20.1.0/28 via 10.20.1.1 dev ens4
10.20.1.1 dev ens4 scope link
192.168.10.0/28 via 192.168.10.1 dev ens5
192.168.10.1 dev ens5 scope link
192.168.20.0/28 via 192.168.10.1 dev ens5
```

建立 IP 資料表規則

在 Cloud Shell 中驗證目前的 IP 表格。

```
sudo iptables -t nat -L -n -v
```

範例：

```

user@fox:~$ sudo iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source      destination

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source      destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source      destination

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source      destination

```

在 Cloud Shell 中更新 IP Tables。

```

sudo iptables -t nat -A POSTROUTING -o ens5 -j MASQUERADE
sudo sysctl net.ipv4.ip_forward=1

```

在 Cloud Shell 中驗證更新後的 IP 資料表。

```

sudo iptables -t nat -L -n -v

```

範例：

```

user@fox:~$ sudo iptables -t nat -L -n -v
Chain PREROUTING (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source      destination

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source      destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source      destination

Chain POSTROUTING (policy ACCEPT 0 packets, 0 bytes)
  pkts bytes target      prot opt in      out     source      destination
    0      0 MASQUERADE  all  --  *        ens5     0.0.0.0/0    0.0.0.0/0

```

15. 更新路由表

在生產者虛擬私有雲中，建立前往消費者子網路 192.168.20.0/28 的靜態路由，下一個躍點為內部負載平衡器。建立完成後，任何傳送至目的地 192.168.20.0/28 的封包 (位於 producer-vpc 內) 都會導向內部負載平衡器。

開啟新的 Cloud Shell 分頁，然後更新專案設定。

在 Cloud Shell 中執行下列操作：

```
gcloud config list project
gcloud config set project [YOUR-PROJECT-NAME]
projectid=YOUR-PROJECT-NAME
echo $projectid
```

在 Cloud Shell 中，使用靜態路由更新 producer-vpc 路由表。

```
gcloud beta compute routes create producer-to-cosmo-subnet-1 --project=$projectid --
network=producer-vpc --priority=1000 --destination-range=192.168.20.0/28 --next-hop-ilb=psc-
ilb --next-hop-ilb-region=us-central1
```

16. 確認從 tiger 到 cosmo 的連線是否成功

Curl 驗證

請執行 curl，確認生產端 VM 執行個體 (tiger) 能與消費端執行個體 (cosmo) 通訊。

開啟新的 Cloud Shell 分頁，然後更新專案設定。

在 Cloud Shell 中執行下列操作：

```
gcloud config list project
gcloud config set project [YOUR-PROJECT-NAME]
projectid=YOUR-PROJECT-NAME
echo $projectid
```

在 Cloud Shell 中使用 IAP 登入 tiger 執行個體。

```
gcloud compute ssh tiger --project=$projectid --zone=us-central1-a --tunnel-through-iap
```

從 tiger 執行個體對 cosmo 的 IP 位址執行 curl，該 IP 位址已在教學課程中識別。

```
curl -v <cosmo's IP Address>
```

範例：

```
user@tiger:~$ curl -v 192.168.20.2
*   Trying 192.168.20.2:80...
* Connected to 192.168.20.2 (192.168.20.2) port 80 (#0)
> GET / HTTP/1.1
> Host: 192.168.20.2
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Fri, 09 Jun 2023 03:49:42 GMT
< Server: Apache/2.4.56 (Debian)
< Last-Modified: Fri, 09 Jun 2023 03:28:37 GMT
< ETag: "27-5fda9f6ea060e"
< Accept-Ranges: bytes
< Content-Length: 39
< Content-Type: text/html
<
Welcome to cosmo's backend server !!
```

恭喜！！您已執行 curl 指令，成功驗證從 producer-vpc 到 backend-vpc 的連線。

17. 清理

在 Cloud Shell 中刪除教學課程元件。

```
gcloud compute instances delete cosmo --zone=us-central1-a --quiet

gcloud compute instances delete rabbit --zone=us-central1-a --quiet

gcloud compute instances delete fox --zone=us-central1-a --quiet

gcloud compute instances delete tiger --zone=us-central1-a --quiet

gcloud compute network-attachments delete psc-network-attachment --region=us-central1 --quiet

gcloud compute firewall-rules delete allow-ingress-to-cosmo allow-tiger-ingress ilb-health-checks ssh-iap-consumer ssh-iap-producer --quiet

gcloud beta compute routes delete producer-to-cosmo-subnet-1 --quiet

gcloud compute forwarding-rules delete psc-ilb --region=us-central1 --quiet
```

```
gcloud compute backend-services delete psc-interface-backend --region=us-central1 --quiet
```

```
gcloud compute instance-groups unmanaged delete psc-interface-instances-ig --zone=us-central1-a --quiet
```

```
gcloud compute health-checks delete hc-http-80 --quiet
```

```
gcloud compute networks subnets delete cosmo-subnet-1 prod-subnet prod-subnet-2 prod-subnet-3
intf-subnet --region=us-central1 --quiet

gcloud compute routers delete cloud-router-for-nat --region=us-central1 --quiet

gcloud compute routers delete cloud-router-for-nat-producer --region=us-central1 --quiet

gcloud compute networks delete consumer-vpc --quiet

gcloud compute networks delete producer-vpc --quiet

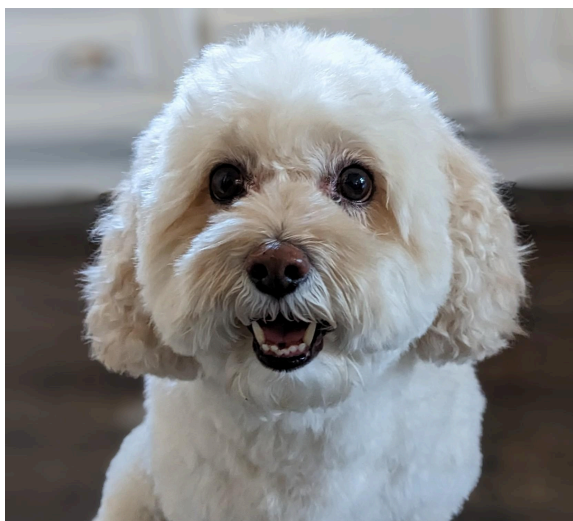
gcloud compute networks delete backend-vpc --quiet
```

18. 恭喜

恭喜！您已成功設定及驗證 Private Service Connect 介面，並透過虛擬私有雲對等互連驗證消費者和生產者連線。

您已建立消費者基礎架構，並新增網路附件，讓生產者可以建立多重 NIC VM，以橋接消費者和生產者通訊。您已瞭解如何透過生產者虛擬私有雲中的內部負載平衡器和靜態路由，使用 PSC 介面透過 VPC 對等互連與第一方/第三方服務通訊。

Cosmopup 認為教學課程很棒！



後續步驟

查看一些教學課程...

- [使用 Private Service Connect 發布及使用 GKE 服務](#)
- [使用 Private Service Connect 發布及使用服務](#)
- [透過 Private Service Connect 和內部 TCP Proxy 負載平衡器，經由混合式網路連線至地端部署服務](#)

延伸閱讀和影片

- [Private Service Connect 總覽](#)
- [什麼是 Private Service Connect？](#)
- [支援的負載平衡器類型](#)

參考文件

- [內部 HTTP\(S\) 負載平衡器總覽](#)
 - [健康檢查總覽](#)
 - [如何使用 Private Service Connect 發布服務](#)
 - [Private Service Connect - 自動 DNS](#)
-